Project Portfolio Governance & Performance Management

Full-Stack Development | Web Analytics | AI-Assisted Engineering | Business Automation

THE OBJECTIVE

Build an **end-to-end working business application**, not just UI screens or disconnected demonstrations. Your solution must connect **frontend -> backend/API -> relational SQL database**, enforce access rules on the server, and produce live management intelligence. We value a candidate who fully owns their implementation, **not an AI-generated application that you cannot explain.**

ASSESSMENT ITEM	EXPECTATION
Interview	**30-60 minutes**, including working demonstration, code walkthrough and an unannounced technical challenge.
Data	**No datasets or CSV files are supplied.** Generate your own credible synthetic data and CSV exports.
Technology	Choose an appropriate modern frontend, backend and server-based relational SQL database.
Environment	**Free-tier hosting** (Cloudflare, AWS, Azure or similar) is encouraged. **Local deployment is equally acceptable**; no paid services are required.
Evaluation emphasis	**Full-stack development is the principal assessment area**, including application design, APIs, persistence, authorization, financial logic and web analytics.

1 Business scenario and outcomes

An internal Project Management Office (PMO) oversees technology delivery initiatives. Teams use spreadsheets, emails, individual trackers and presentations for project updates, expenses, milestones, risks and approvals. Leadership has no trusted single view of delivery health, cost exposure, overdue work or project performance.

You have been engaged to design and develop a **Project Portfolio & Financial Control Portal**. It must help operational teams maintain project records while allowing management to explore reliable, data-driven insights through a separate interactive web dashboard.

Demonstrable business journeys

1. A project manager creates a project, assigns an owner and records its delivery dates and initial approved budget.
2. A team member updates an assigned task. The persisted record changes project progress and the executive dashboard.
3. A manager updates a milestone or risk, with clear project-level visibility and appropriate validation.
4. An authorized user records and posts a cost. **Actual spend, remaining budget and utilization** update from SQL.
5. A change request is submitted and reviewed through a controlled approval workflow with history.
6. A management user filters the portfolio dashboard, opens a KPI drill-down and traces its value back to database records.

2 Required application modules

The modules must share users, project identifiers, database relationships and business rules. A collection of disconnected pages, hardcoded samples or screenshots does not qualify.

MODULE	MINIMUM FUNCTIONAL REQUIREMENTS
01 Project portfolio	Create, view and edit projects. Search/filter by department, owner, status and dates. Provide a project details view and basic delivery status.

MODULE	MINIMUM FUNCTIONAL REQUIREMENTS
02 Tasks and milestones	Create/update tasks, assignees, status, priority, dates, weights and progress. Manage milestones. Indicate overdue/upcoming work and calculate project progress.
03 Risks and issues	Record type, owner, probability, impact, mitigation, due date and status. Expose score = probability x impact and show open high-exposure items.
04 Change requests and approvals	Submit project-linked changes, review, approve/reject/return with decision reason. Enforce permitted transitions, self-approval prevention and durable audit history.
05 Financial management	Maintain approved project budgets, revisions, cost categories, transaction entries, monthly forecasts, financial summaries and budget-overrun alerts. Details in Section 4.
06 **Executive web dashboard**	Custom responsive browser dashboard with real SQL/API-backed KPIs, cross-filters, analytics visuals and drill-downs, including financial intelligence. Details in Section 5.
07 User access and activity	Demonstrate at least Contributor, Project Manager and Administrator roles, secure sign-in, scoped access and selected audit/activity events.

ONE CONNECTED SOLUTION

The approval workflow is only one module. The core of this assignment is the engineering of a coherent project-management and financial application. The **web analytics dashboard must be developed as a real application**, not embedded Power BI, static charts or a visual prototype.

3 Full-stack engineering requirements | Primary focus

3.1 Frontend engineering

- **Responsive application shell:** coherent navigation, dashboard and detail pages, consistent typography, usable desktop layouts and practical mobile behavior.
- **Real forms and state management:** controlled inputs, create/edit flows, validation feedback, confirmation for consequential actions, loading/empty/error states and successful save feedback.
- **Data interaction:** search, filter, sort and sensible pagination for project or task lists; refresh displayed state after mutations without requiring a full application restart.
- **Reusable architecture:** clearly structured pages/components/services; explain routing, API client, state approach and reusable form/table/chart components.
- **UX and accessibility:** clear status/priority indications, keyboard-usable controls, readable contrast and understandable errors. Visual polish must never disguise broken functionality.

3.2 Backend and API engineering

- **Working API layer:** implement real endpoints for projects, tasks/milestones, risks, approvals, financial transactions and dashboard aggregates. Provide a concise route list or OpenAPI collection.
- **Business logic on the server:** enforce workflow transitions, assignment rules, finance validations, role restrictions and calculated metrics in backend services/queries, not only in JavaScript.
- **HTTP fundamentals:** use appropriate methods, status codes, request models, input validation and predictable structured error messages.
- **Reliable persistence:** all create/update actions are committed to the SQL database; no in-memory-only storage or hardcoded frontend arrays for business records.
- **Transaction integrity:** approval decisions and their history, or a posted expense and its audit entry, must commit atomically. Explain how partial failures are prevented.
- **Maintainability:** sensible separation between controllers/routes, domain services, repository/data access and frontend code. Demonstrate modularity rather than one oversized source file.

3.3 Relational database design

Create an appropriate relational model using **SQL Server, PostgreSQL, MySQL or an equivalent server-based SQL database**. Include at least the entities below or documented equivalents.

ENTITY GROUP	DESIGN EXPECTATIONS
People / portfolio	Departments, Users, Projects and project ownership or membership.

ENTITY GROUP	DESIGN EXPECTATIONS
Delivery	Tasks, Milestones, Risks/Issues and optional progress snapshots.
Governance	ChangeRequests and ChangeHistory/AuditEvents.
Finance	ProjectBudgets, BudgetRevisions, CostCategories, CostTransactions and FinancialForecasts.
Automation / logging	AutomationRuns, Outbox or notification/audit records for executed events.

- Show **primary keys, foreign keys, uniqueness, referential integrity, nullable relationships and indexes** appropriate for filtering and joins.
- Provide schema creation scripts or migrations and explain relationship cardinalities, including the risk of joining multiple one-to-many tables.
- Use parameterized queries or a correctly configured ORM. Demonstrate at least one **non-trivial SQL aggregation** and one data validation or database constraint.
- Distinguish source-of-truth transactional records from derived KPIs. Changes entered in the application must survive a restart.

3.4 Authentication, role-based authorization and security

ROLE	ILLUSTRATIVE ACCESS POLICY
Contributor	Read assigned/permitted work; update own task progress; submit change requests. No approval or posting of project financial transactions.
Project Manager	Manage assigned projects, tasks, milestones, risks and approved finance operations. Decide eligible change requests but **never approve own request**.
Administrator	Cross-portfolio administration, role management (or seeded role setup), approved financial configuration and audit review.

- **Authenticate real demo users** via an identity provider or securely handled development accounts. A role selector or hidden UI button is not authentication.
- **Authorize every sensitive backend route and resource**. Prevent cross-project access and ID-based escalation, not merely missing menu items.
- Demonstrate that a **Contributor calling the approval or financial posting API directly is denied** even if the frontend is bypassed.
- Protect secrets outside source control, handle invalid inputs defensively and explain the mitigations used for SQL injection, unsafe tokens, insecure cookies or CORS as applicable.

3.5 Quality, reliability and delivery

- Include meaningful server logging and error handling, with a correlation/request identifier or an equivalent means to investigate failures.
- Show one concurrency scenario (for example optimistic version checking or a documented conflict strategy) and how double submissions are prevented.
- Supply at least **five executable automated checks** across API business logic, RBAC, financial calculations or workflow rules, plus a manual end-to-end test checklist.
- Use Git with a meaningful commit history, setup instructions, environment-variable example and database migration/import process. Docker Compose is welcome but optional.
- Explain application structure, choices and tradeoffs. The interviewer may ask for a live modification in any source file.

4 Financial management module | Required

Treat finance as an operational module with editable business records and defensible calculations, **not simply a financial chart on the dashboard**. Currency may be AED; all sample amounts must be fictional.

CAPABILITY	WHAT MUST WORK
Budget management	Store an initial approved budget per project, display current approved budget, and record dated budget revisions with authorization and rationale.
Cost transactions	Add and view dated expenses by project/category. Validate positive amounts and required fields. Distinguish draft from **posted** costs, or clearly document an equivalent approval rule.

CAPABILITY	WHAT MUST WORK
Cost tracking	Calculate **actual spend, remaining budget, utilization percentage**, spending by category and monthly spend trends directly from SQL.
Forecasting	Capture monthly future estimates or forecast-to-complete. Calculate **estimate at completion (EAC)** and projected budget variance with a documented rule.
Controls and audit	Restrict financial mutations to authorized users; capture actor/time and prevent unauthorized posting or silent deletion of posted transactions.
Financial insight	At least one budget-overrun or overspend alert, project comparison, and drill-down from a financial KPI to its underlying cost transactions.

Financial definitions to apply consistently

- **Current approved budget = initial approved budget + approved budget revisions.** Pending or rejected revisions have no effect.
- **Actual spend = total of posted cost transactions** on or before the reporting cutoff. Draft/unposted transactions are excluded.
- **Remaining budget = current approved budget - actual spend. Budget utilization = actual spend / current approved budget**, with sensible zero-budget handling.
- **EAC = actual spend + forecast-to-complete** under your documented remaining-period forecast method. Projected variance = current approved budget - EAC.
- **Approved change requests do not automatically become posted expenditure or budget revisions.** Any link between changes and budgets must be explicit and auditable.

5 Custom data-driven executive dashboard

Build a **web-based management dashboard as part of your portal**. It must retrieve live, scope-aware values from backend analytics endpoints backed by relational SQL queries. **No hardcoded KPI totals, Excel-only data models, screenshots or Power BI iframes.**

Required KPI cards and visualizations

- **Delivery:** total/active projects, portfolio task completion/progress, overdue incomplete tasks and milestones due within 14 days.
- **Risk and governance:** open high-exposure risks/issues, blocked work and pending change requests.
- **Financial:** approved budget, booked/actual spend, remaining budget, utilization percentage, forecast EAC and projected overrun count or amount.
- **At least five meaningful interactive visuals** spanning delivery and finance, such as project health, milestone roadmap, overdue work by owner, risk matrix, budget vs actual vs EAC, monthly cost trend or spend by category.
- **At least four coordinated filters** drawn from department, project, manager, lifecycle status, timeframe and cost category (category can apply only to compatible finance visuals).
- **Two drill-down interactions** at minimum, including **one finance KPI/chart opening underlying transactions** and one delivery KPI/chart opening filtered tasks, milestones or risks.
- Responsive chart behavior, obvious AED/% units, clear legends, consistent period selection and proper loading/empty/error states.

Mandatory analytics integrity demonstration

The dashboard must recompute from backend/SQL after **(1) a task progress update and (2) a newly posted expense**, with no client-side KPI patching. Explain the source tables, filter context and calculation used for two KPI values. Avoid inflating budget or spend through incorrect joins across tasks, risks and costs.

> **SEPARATE REPORTING PRODUCTS**
>
> **The custom web dashboard and the Power BI report are two different deliverables.** They may reuse the same SQL database and business definitions, but the browser dashboard must have its own application components, API calls and interactive behavior.

6 Create your own synthetic data and CSVs

No prepared CSVs, Excel sheets or sample database are provided. Create a reproducible **data-generation script** (SQL, Python, JavaScript or similar), generate realistic fictional records, export the generated tables to CSV, and import the same records into your relational SQL database.

DATA FAMILY	SUGGESTED MINIMUM SIZE
Organizations and users	At least **3 departments** and **15 demo users**, distributed across roles.
Projects and delivery	At least **8 projects**, **80 tasks** and **24 milestones** across different phases and owners.
Risks and governance	At least **20 risk/issue records**, **15 change requests** and corresponding change audit events.
Financial data	At least **80 cost transactions**, several budget revisions, **4 cost categories** and monthly future forecasts across projects.
Progress and automation	Several historical progress snapshots per project and enough automation log records to evidence workflow behavior.

Required edge cases intentionally present in your data

- **At least one project with no posted spend**, one nearing its budget and one forecast to exceed its current approved budget.
- **At least five overdue incomplete tasks**, a task due exactly on the reporting date, one unassigned task and at least one completed project.
- A closed high-exposure risk that must not count as open, a high-exposure open risk, and one returned or rejected change request.
- An approved budget revision and an unapproved revision, posted and draft costs, at least one cost after the reporting cutoff, and one project without forecast data (explain treatment).
- Records from more than one department and date period, enabling genuine filtering and drill-down.

Use a **fixed reporting snapshot of 30 September 2026** for baseline demonstrations; explain how production would use today or a user-selected date. Your generator must support repeatable output (for example a fixed random seed). Submit **the generator, generated CSVs, schema/import scripts and a brief data dictionary**. Do not use actual employees, customers, vendors, credentials or company data.

7 SQL and Power BI | Independent reporting track

Create an actual Power BI report connected to your **relational SQL database**, not directly to the generated CSV files. It should make independent use of data transformation, modeling, measures and dashboard design.

- Demonstrate a sound relational model with projects, tasks, risk/issue records and finance facts; include appropriate calendar/date treatment and explain relationship/filter direction.
- Prepare and explain at least **four practical SQL queries**: overdue tasks, project financial performance, high-exposure open risks and period spend/forecast or milestone status.
- Create **at least five explicit DAX measures** including overdue count, task progress or completion, approved budget, actual spend and budget utilization or forecast variance.
- Create at least one polished interactive Power BI management page with relevant slicers, business-readable units and charts.
- **Reconcile at least four common KPIs** across SQL results, web dashboard and Power BI for the identical snapshot/filter scope, explaining discrepancies.

8 Business automation and MS Access / VBA (or any other appropriate tech. stack)

8.1 One executed business automation

Implement one working automation using Power Automate, a backend worker, Python, UiPath, a scheduled task or another appropriate technology. Examples: **approved-change notifications**, project overdue escalations, expense posting alerts or month-end financial summaries.

- Demonstrate trigger, rules, a real run, visible outcome and persistent execution log/outbox record.
- Show safe retry/failure behavior and protection against sending the same notification or posting the same event twice.
- An in-app notification or local event outbox is sufficient. Live corporate email is not required.

8.2 Compact MS Access / VBA evidence (Optional)

Create a small **.accdb** utility using at least two linked or imported related SQL tables, a functional form, saved query, useful report and one executable VBA button (for example requery, filtered export or exception listing). Demonstrate basic error handling and explain how Access relates to your main SQL source. This is a **focused skills check**, not a second application.

9 AI-assisted development and technical ownership

AI IS ENCOURAGED, UNDERSTANDING IS MANDATORY

We value AI as a development accelerator when combined with genuine engineering judgement. Your submission should be a **working application you can explain, diagnose, test and modify**, **not an AI-generated application that you cannot explain.**

- Provide an **AI Development Log with at least four meaningful prompt iterations**. Include requirement/problem, prompt, AI result, what you accepted/rejected, revised implementation and test evidence.
- Include at least one example where AI was **wrong, insecure, inefficient or analytically incorrect**, and show how you found and corrected it independently.
- Describe your prompt strategy for frontend architecture, backend business rules, database constraints, financial calculations or debugging.
- Be ready to open any relevant file, follow an API request end to end, explain the SQL or DAX and make an unannounced small change during the interview.
- Disclose which sections were AI-assisted, what you personally validated, where you changed the generated code and why.

10 Testing, security and deployment acceptance

Provide Git history, reproducible local setup, **secret-free configuration**, SQL migrations/imports and demo logins. If hosted on a free tier, share the **live URL** and deployment steps. In both cases, the working frontend, API and SQL database must be verifiable.

LIVE ACCEPTANCE TEST	EXPECTED EVIDENCE
A End-to-end persistence	Create a project/task; edit its progress; reload; verify SQL and browser state changed.
B Finance mutation	Post a new cost using an authorized account; demonstrate recalculated spend, remaining budget and financial drill-down.
C Approval transaction	Review a valid change and show the decision plus exactly one corresponding history entry.
D Authorization denial	Call approval and expense-posting endpoints as a Contributor; backend refuses both with clear responses.
E Dashboard integrity	Use filters and drill-downs; reproduce representative KPI values with SQL; ensure charts update after edits.
F Analytics agreement	Reconcile four shared metrics among SQL, the browser dashboard and Power BI.
G Automation/VBA	Show a triggered automation run and one working Access form/VBA action.
H Engineering quality	Show five automated checks and an error log; explain an edge case or a failure you corrected.

11 Required submission and interview presentation

Submission checklist

- **Working project source code:** frontend, backend, tests, Git history, local setup and a hosted URL if available.
- **Data and API package:** SQL schema/migrations, data generator, **candidate-generated CSVs**, import commands, documented entities and API route list/collection.
- **Reporting:** working web dashboard within the application; Power BI **.pbix**, commented SQL queries and cross-system KPI reconciliation notes.
- **Supporting deliverables:** focused Access **.accdb**, automation implementation and run evidence, AI Development Log and architecture/security notes.
- **Management presentation: 6 to 8 PowerPoint slides** covering business value, solution overview, architecture, financial management, web intelligence, automation, AI approach, testing and delivery.

Optional enhancements

If your required modules work reliably, you may add a compact roadmap/Gantt, task dependencies, multi-user edit conflict handling, Excel/PDF exports or enhanced dashboards. **Optional features do not compensate for weak backend engineering, incorrect financial calculations or missing server-side authorization.**

FINAL SELECTION PRINCIPLE

A smaller, well-designed and **fully working vertical slice** with explainable engineering decisions will score better than a visually impressive portal with fake integrations, incorrect financial KPIs or nonfunctional APIs.